

Tache: CLI Task Management System with Local AI

TypeScript Advanced & System Design Basics

Purple Ram August 09, 2026

Assignment Overview

Document Contents:

1. Introduction	3
2. Background	3
3. Methodology	4
3.1. Project Definition	4
3.2. Technology Stack	4
3.3. Software Architecture	4
3.4. Domain Model	5
3.5. Advanced TypeScript	6
3.6. Design Patterns	6
3.7. CLI Interface	6
3.8. Database and Search	7
3.9. Scheduling and Notifications	7
3.10. Local AI Integration	7
4. Implementation	8
4.1. CRUD Operations	8
4.2. Undo and Redo	9
4.3. Import and Export	9
4.4. Productivity Metrics	9
5. Testing and Quality Assurance	9
6. Git Workflow	10
7. Results	10
7.1. Performance and Maintainability	11
8. Discussion	11
9. Deliverables	11
10. Bonus Implementation	12
10.1. Custom ESLint Plugin	12
10.2. Structured Logging	12
11. Bonus [01]: Systems Thinking Applied to Tache	12
11.1. Objective	12

Assignment Details:

- **Course:** The Blachole
- **Instructor:** Documentation and handbooks
- **Due Date:** August 08, 2026
- **Hardware:** PC
- **Software:** TypeScript, Node.js/Bun, SQLite via Drizzle ORM, Ollama, Vitest, Commander.js, Zod, GitHub Actions
- **Duration:** ~five days
- **Lab Number:** Lab 01

11.2. Relationship with the Main Project ...	13
11.3. Part A – System Model	13
11.4. Part B – Architectural Decisions	14
11.5. Part C – Systems Thinking	14
11.6. Part D – Architecture Evolution	15
11.7. Part E – Bottleneck Analysis	16
11.8. Part F – Architecture Fitness Functions	16
11.9. Part G – Architecture Review	17
11.10. Deliverables	17
11.11. Architectural Decision Record	17
11.12. Expected Outcome	18
12. Conclusion of Bonus	19
13. Conclusion	19

Introduction

This lab explores advanced TypeScript concepts and fundamental software architecture principles through the development of a production-oriented command-line task management system named **Tache**.

The project is designed to move beyond basic CRUD application development by combining strongly typed application code, software design patterns, SOLID principles, clean architecture, automated testing, continuous integration, and local artificial intelligence.

The central objective is to develop a **CLI** application that could realistically be used as a daily developer productivity tool. The system manages tasks with priorities, deadlines, tags, dependencies, scheduling information, and productivity metrics while maintaining persistent data in SQLite through **Drizzle ORM**.

A key differentiator of the project is the integration with **Ollama**, allowing the application to generate intelligent task-prioritization suggestions using a locally running language model without requiring an external API key or cloud-based AI service.

The objectives of this lab are:

1. Apply advanced TypeScript features including Generics, Conditional Types, Mapped Types, and Template Literal Types
2. Apply common software design patterns in a practical system
3. Understand and apply SOLID principles
4. Design an application using Clean Architecture
5. Implement persistence with SQLite and Drizzle **ORM**
6. Build a professional command-line interface using Commander.js
7. Integrate a locally hosted AI model through **Ollama**
8. Achieve at least 90% automated test coverage using **Vitest**
9. Implement a professional Git workflow using conventional commits and semantic versioning
10. Understand the architectural trade-offs involved in building maintainable software

Background

Modern **TypeScript** applications benefit from more than a collection of working functions. As applications grow, maintainability, testability, extensibility, and separation of concerns become increasingly important.

TypeScript provides advanced type-system capabilities that can be used to encode domain rules directly into the source code. Generics allow reusable abstractions, Conditional Types enable type-level decisions, Mapped Types allow systematic transformations of existing types, and Template Literal Types make it possible to construct constrained string types.

These features become particularly useful when combined with established software engineering principles.

The project therefore applies four major design patterns:

1. **Factory Pattern** for controlled creation of domain and infrastructure objects
2. **Singleton Pattern** for shared application-level resources where appropriate
3. **Observer Pattern** through Node.js *EventEmitter* for real-time task notifications
4. **Strategy Pattern** for interchangeable prioritization algorithms

The project also follows the SOLID principles:

1. **Single Responsibility Principle** — each component should have one primary responsibility
2. **Open/Closed Principle** — components should be extensible without unnecessary modification

3. **Liskov Substitution Principle** — implementations should remain substitutable for their abstractions
4. **Interface Segregation Principle** — interfaces should remain focused and specific
5. **Dependency Inversion Principle** — high-level application logic should depend on abstractions rather than infrastructure details

Methodology

Project Definition

The system was defined as a command-line task management application intended for regular developer use.

The application must support:

1. Task creation, modification, deletion, and retrieval
2. Priorities and deadlines
3. Tags and task dependencies
4. Task scheduling
5. Full-text search
6. Import and export
7. Undo and redo operations
8. Productivity metrics
9. Real-time notifications
10. Local AI-based prioritization suggestions

The application must also be distributed as an NPM package and supported by automated tests and CI/CD infrastructure.

Technology Stack

The following technologies were selected:

1. **TypeScript** for the primary programming language
2. **Node.js/Bun** as the JavaScript runtime
3. **SQLite** for lightweight relational persistence
4. **Drizzle ORM** for type-safe database access
5. **Zod** for runtime validation and schema definitions
6. **Commander.js** for CLI command handling
7. **Ollama** for local AI inference
8. **Vitest** for unit and integration testing
9. **GitHub Actions** for CI/CD automation
10. **ESLint** for static analysis and code quality

Software Architecture

The application follows a Clean Architecture structure:

```
src/
├── domain/
│   ├── entities/
│   ├── value-objects/
│   ├── repositories/
│   └── services/
├── application/
│   ├── commands/
│   ├── queries/
│   └── use-cases/
├── infrastructure/
│   ├── database/
│   ├── filesystem/
│   ├── ollama/
│   └── scheduling/
├── presentation/
│   └── cli/
└── shared/
    ├── types/
    ├── errors/
    └── utilities/
```

The **domain** layer contains business entities and rules and must remain independent from external infrastructure.

The **application** layer implements use cases and provides a basic *CQRS* separation between commands that modify state and queries that retrieve information.

The **infrastructure** layer contains concrete implementations for SQLite, file-system operations, scheduling, and communication with *Ollama*.

The **presentation** layer provides the command-line interface through Commander.js.

The **shared** layer contains reusable types, errors, and utilities that do not belong to a specific architectural layer.

This separation makes it possible to replace infrastructure components without rewriting the core business logic.

Domain Model

The central entity is the Task.

A task contains, at minimum:

1. Unique identifier
2. Title
3. Description
4. Priority
5. Status
6. Due date
7. Tags
8. Dependencies
9. Creation timestamp
10. Completion timestamp

Tasks are persisted in SQLite while domain-specific validation remains independent of the database implementation.

Runtime input validation is performed through **Zod** schemas before data enters the application layer.

Advanced TypeScript

Advanced TypeScript features are used to improve correctness and reduce duplicated definitions.

Generics are used to create reusable repository and service abstractions.

Conditional Types are used where application types depend on compile-time conditions.

Mapped Types are used to derive variations of domain models, such as update **DTOs** and read-only representations.

Template Literal Types are used for constrained identifiers and event names.

For example:

```
type TaskStatus = "pending" | "in-progress" | "completed";

type TaskEvent<T extends TaskStatus> =
  `task:${T}`;

type TaskEvents =
  | TaskEvent<"pending">
  | TaskEvent<"in-progress">
  | TaskEvent<"completed">;
```

This approach allows the compiler to validate event names and reduces the possibility of invalid strings being introduced into the application.

Design Patterns

The **Factory Pattern** is used to centralize creation of objects whose construction may involve validation or configuration.

The **Strategy Pattern** is used for task prioritization. Different strategies can be implemented without changing the use case responsible for requesting a prioritization operation.

Possible strategies include:

1. Deadline-based prioritization
2. Historical productivity prioritization
3. Dependency-based prioritization
4. AI-assisted prioritization

The **Observer Pattern** is implemented using Node.js *EventEmitter* to notify subscribers when task-related events occur.

The **Singleton Pattern** may be applied to resources such as application configuration or controlled shared infrastructure instances, while avoiding unnecessary global state.

The **Command Pattern** is used to implement undo and redo functionality.

CLI Interface

Commander.js provides the command-line interface.

Representative commands include:

```

tache task add "Implement authentication"
tache task list
tache task show <id>
tache task update <id>
tache task delete <id>
tache task complete <id>

tache search "authentication"

tache export --format json
tache export --format csv
tache export --format markdown

tache ai prioritize
tache stats
tache undo
tache redo

```

The **CLI** acts exclusively as a presentation layer and delegates business operations to application use cases.

Database and Search

SQLite is used as the persistence layer because the application is designed primarily as a local developer tool.

Drizzle ORM provides a type-safe interface between the TypeScript application and the database.

SQLite FTS5 is used to implement full-text search over task information.

This allows users to search task titles, descriptions, and other indexed textual fields without loading the entire task collection into memory.

Scheduling and Notifications

Task scheduling is implemented using Node.js or Bun scheduling mechanisms.

Scheduled operations can trigger application events, which are propagated through the Observer Pattern.

For example:

```

Scheduler
|
v
Task deadline reached
|
v
EventEmitter
|
+----> CLI notification
|
+----> Productivity metrics
|
+----> Logging

```

This architecture keeps scheduling independent from notification consumers.

Local AI Integration

The AI component uses **Ollama** with a locally hosted model such as llama3.

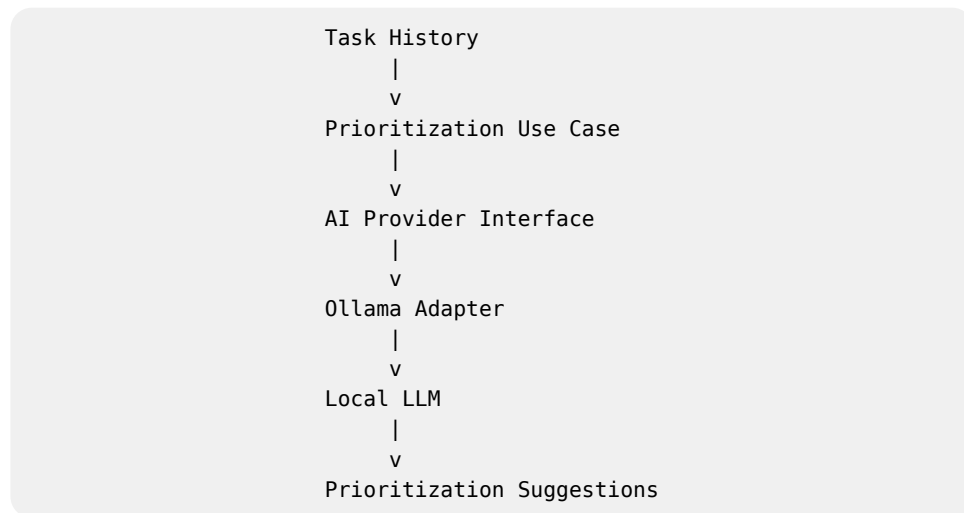
The application does not require an external API key.

Historical task information can be used to generate prioritization suggestions based on patterns such as:

1. Completion frequency
2. Missed deadlines
3. Task categories
4. Historical priorities
5. Average completion time
6. Current workload

The AI component is treated as an infrastructure dependency and accessed through an abstraction so that the domain and application layers do not depend directly on **Ollama**.

A simplified flow is:

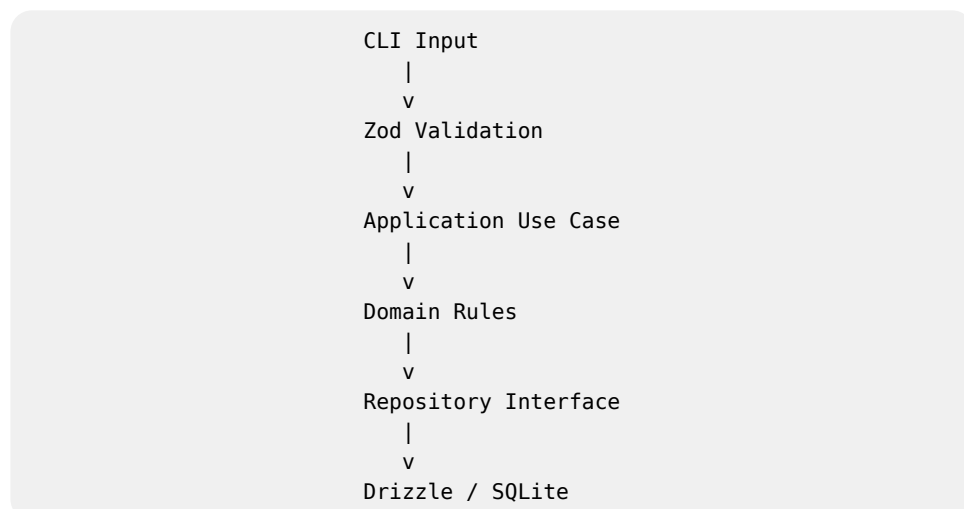


Implementation

CRUD Operations

The application implements complete CRUD functionality.

Every command follows the same general flow:



This structure ensures that the **CLI** and **database** implementation do not contain business rules.

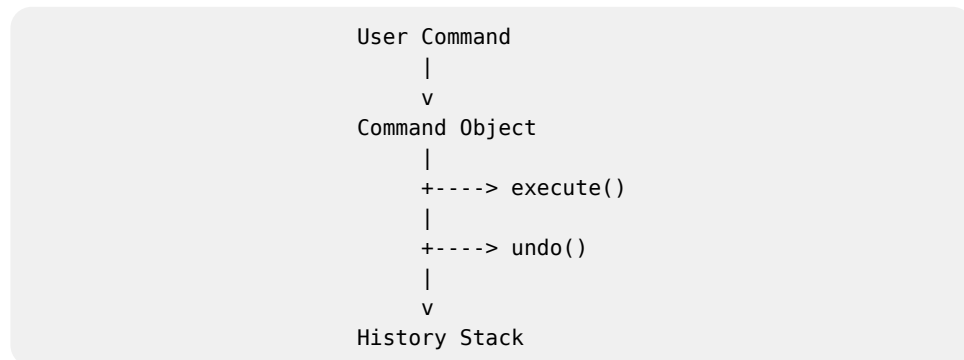
Undo and Redo

Undo and **redo** operations are implemented using the Command Pattern.

Each state-changing operation is represented as a command containing the information required to execute and reverse the operation.

An unlimited history is maintained, allowing users to reverse previous task modifications.

The architecture can be represented as:



Import and Export

The application supports data exchange in three formats:

1. JSON
2. CSV
3. Markdown

Import operations validate incoming data before it reaches the domain layer.

Export operations transform domain entities into the requested representation without changing the underlying domain model.

Productivity Metrics

The system calculates productivity metrics such as:

1. Tasks completed per day
2. Current and historical streaks
3. Average task completion time
4. Tasks completed by category
5. Deadline completion rate

These metrics provide users with historical information that can also serve as input for AI-assisted prioritization.

Testing and Quality Assurance

Vitest is used as the primary testing framework.

The project targets at least 90% test coverage.

Testing is divided into several levels:

1. Unit tests for domain entities and value objects
2. Unit tests for application use cases
3. Integration tests for repositories
4. Integration tests for SQLite persistence
5. Tests for CLI commands
6. Tests for import and export functionality
7. Tests for AI adapters
8. End-to-end tests for critical workflows

External dependencies are isolated through interfaces and test doubles where appropriate.

The CI pipeline automatically executes the test suite and verifies the required coverage threshold.

Git Workflow

A professional Git workflow is used throughout development.

Commits follow the Conventional Commits specification:

```
feat: add task prioritization
fix: prevent invalid task dependencies
test: add repository integration tests
refactor: isolate task scheduling service
docs: update architecture documentation
chore: configure CI pipeline
```

Semantic Versioning is used to communicate the impact of releases:

```
MAJOR.MINOR.PATCH
```

Development follows a branching strategy that separates feature development from stable releases.

GitHub Actions automatically executes quality checks, tests, coverage validation, and package-related workflows.

Results

The expected result is a functional CLI application capable of managing a complete local task workflow.

The completed system should demonstrate:

1. Strong static typing through advanced TypeScript
2. Clear separation between domain, application, infrastructure, and presentation
3. Correct implementation of selected design patterns
4. Practical application of SOLID principles
5. Persistent SQLite storage
6. Full-text search
7. Import and export capabilities
8. Undo and redo functionality
9. Scheduled task operations

10. Real-time event notifications
11. Local AI prioritization through Ollama
12. At least 90% automated test coverage
13. Automated CI/CD validation
14. NPM package publication

Performance and Maintainability

The architecture prioritizes maintainability over premature optimization.

SQLite is sufficient for the expected workload of a local CLI application, while FTS5 provides efficient text search without requiring an external search engine.

The use of interfaces around infrastructure dependencies also allows individual components to be replaced without affecting the core application.

Vitest provides a fast feedback cycle during development, making it suitable for a project with a large automated test suite.

Discussion

The project demonstrates that a relatively small command-line application can be designed using principles normally associated with larger production systems.

Advanced TypeScript features provide compile-time guarantees, while Zod complements them with runtime validation. This distinction is important because TypeScript types disappear during execution and therefore cannot independently validate external input.

Clean Architecture establishes boundaries between **business logic** and implementation details. This makes the system easier to test and allows infrastructure such as SQLite or Ollama to be replaced without rewriting the core use cases.

The **Strategy Pattern** is particularly useful for the prioritization subsystem because different prioritization algorithms can evolve independently.

The **AI integration** also demonstrates an important architectural principle: external services should be treated as replaceable infrastructure rather than embedded throughout the business logic.

The project also highlights several trade-offs.

A Clean Architecture implementation introduces additional abstractions and files compared with a simple CRUD application. This increases initial development complexity but can significantly reduce coupling as the system grows.

Similarly, using a local LLM avoids API costs and external dependencies but introduces constraints related to model performance, local hardware resources, and response latency.

Deliverables

The project must produce the following deliverables:

1. Functional NPM package containing the Tache CLI
2. Professional README with CI, coverage, and version badges
3. Automated GitHub Actions CI/CD pipeline

4. At least 90% test coverage
5. Semantic versioning and automated CHANGELOG
6. DEV.to article explaining the architecture and major design decisions
7. Five-minute YouTube demonstration
8. LinkedIn post documenting the main technical learnings

Bonus Implementation

Two optional extensions may be implemented.

Custom ESLint Plugin

A custom *ESLint* plugin can be developed to enforce project-specific coding conventions and architectural rules.

Possible rules include:

1. Preventing infrastructure imports inside the domain layer
2. Enforcing naming conventions
3. Restricting direct database access outside infrastructure
4. Detecting forbidden dependencies between architectural layers

Structured Logging

Structured logging can be implemented using Pino.

The logging system should support configurable levels such as:

```
trace
debug
info
warn
error
fatal
```

Logs should contain structured metadata rather than relying exclusively on formatted strings.

Bonus [01]: Systems Thinking Applied to Tache

Objective

The objective of this bonus is to apply **Systems Thinking** and **Software Architecture** principles directly to the main **Tache** project.

Instead of analyzing an unrelated external system, the student will analyze Tache itself as a software system, identifying the relationships between business requirements, architecture, components, data, events, infrastructure, performance, and future evolution.

The activity aims to develop the ability to answer not only:

> “How should this feature be implemented?”

but also:

> “What consequences will this decision have on the system as a whole?”

The focus is to understand Tache not simply as a collection of features, but as an architectural system that must remain maintainable and evolvable over time.

Relationship with the Main Project

The bonus must be developed directly on top of the architecture implemented in the main Tache project.

The analysis must reference actual components of the system, including:

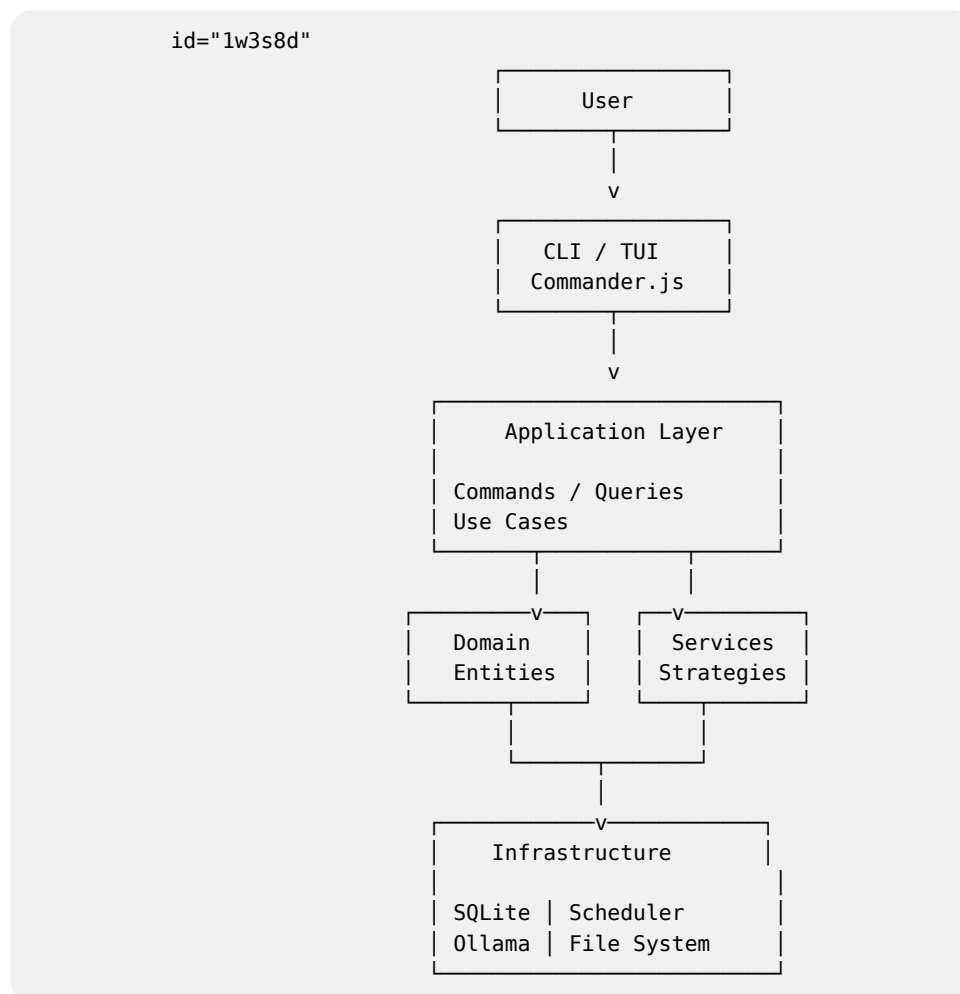
Clean Architecture Domain Layer **Application Layer** SQLite and Drizzle ORM **CLI with Commander.js** EventEmitter **Command Pattern for Undo/Redo** Strategy Pattern for task prioritization **Ollama for local AI** Task Scheduler **SQLite FTS5 full-text search** Import/Export **Productivity Metrics** Testing and CI/CD

The student must identify how these components form a single system and how decisions made in one part of the application affect other parts.

Part A — System Model

The first step is to model Tache as a system.

The student must produce a high-level architecture diagram showing at least:



The diagram must also show the most important data flows and event flows within the system.

Part B — Architectural Decisions

The student must document at least five relevant architectural decisions made during the development of Tache.

For each decision, the following questions must be answered:

What problem needed to be solved? What alternatives were considered? **Which solution was selected?** Why was it selected? **What are its advantages?** What are its disadvantages? **What trade-offs were accepted?** How could the decision affect the system in the future?

Possible decisions to analyze include:

11.4.1. Clean Architecture

Why should domain, application, infrastructure, and presentation be separated instead of implementing the entire application directly inside CLI commands?

11.4.2. SQLite

Why use SQLite instead of PostgreSQL or another database server?

11.4.3. Ollama

Why run the AI model locally instead of using an external API?

11.4.4. EventEmitter

Why use an event-driven notification mechanism?

11.4.5. Strategy Pattern

Why should task prioritization be implemented as a replaceable strategy instead of a single hard-coded function?

11.4.6. Command Pattern

Why should Undo/Redo be implemented through commands?

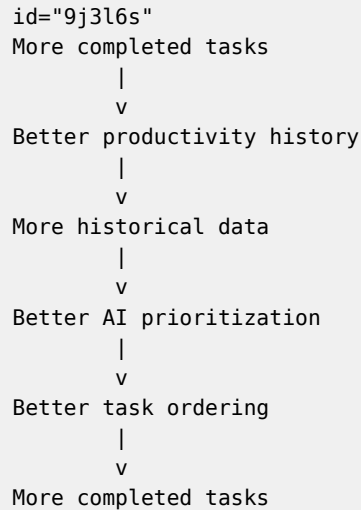
11.4.7. FTS5

Why use SQLite's full-text search capabilities instead of an external search engine?

Part C — Systems Thinking

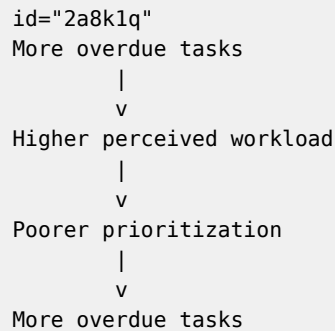
The student must identify at least three cause-and-effect relationships within Tache.

For example:



This represents a potential **positive feedback loop**.

Another example:



The student must identify whether each relationship represents a positive feedback loop, negative feedback loop, or indirect effect.

The analysis should focus on how local changes can produce system-wide consequences.

Part D — Architecture Evolution

Tache is initially designed as a local-first application for a single user.

The student must analyze how the architecture would need to evolve if the requirements changed.

The proposed scenario is:

> Tache becomes a platform used by 10,000 users, with multi-device synchronization, user accounts, and remote data storage.

The analysis must address:

1. What could remain unchanged?
2. What would need to change?
3. Which components would no longer be appropriate?
4. Would SQLite still be sufficient?
5. Would Ollama still run locally?
6. How would synchronization be implemented?
7. How would conflicts be resolved?
8. Would EventEmitter still be sufficient?

9. Would a message broker be necessary?
10. How would data be isolated between users?
11. What new security problems would appear?
12. Which current architectural decisions would become liabilities?

The objective is not to implement this new architecture.

The objective is to demonstrate the ability to predict how current architectural decisions affect future system evolution.

Part E — Bottleneck Analysis

The student must identify at least three potential bottlenecks in Tache.

Possible examples include:

1. SQLite under concurrent workloads
2. Ollama inference latency
3. Full-text search over large datasets
4. Scheduler performance with a large number of scheduled tasks
5. Unlimited Command Pattern history
6. Productivity metric calculations over large datasets
7. Large import/export operations

For each bottleneck, the student must describe:

1. How the problem could appear
2. What would cause it
3. How it could be measured
4. What a possible solution would be
5. What cost or trade-off the solution would introduce

The analysis should distinguish between a theoretical bottleneck and one that is actually observable in the current implementation.

Part F — Architecture Fitness Functions

The student must define at least five rules that help ensure the architecture remains healthy as the project grows.

Examples:

```
id="8v1p3n"
Domain must not import Infrastructure.

Infrastructure must not be required by Domain entities.

CLI commands must not access SQLite directly.

All external AI providers must implement the AIProvider interface.

Application use cases must be testable without a real database.

All state-changing operations must be represented by Commands.
```

Whenever possible, these rules should be enforced automatically through tests, ESLint, or CI.

This section directly connects the bonus with the optional custom ESLint plugin and the main project's quality requirements.

Part G — Architecture Review

After implementing Tache, the student must perform a critical review of the resulting architecture.

The review must answer:

1. What was the best architectural decision?
2. What was the worst architectural decision?
3. Which component became unnecessarily complex?
4. Where is the strongest coupling?
5. Which abstractions proved useful?
6. Which abstractions were unnecessary?
7. Which part of the system would be hardest to replace?
8. Which decision would be made differently if the project were started again?

The answers must be supported by concrete examples from the implemented codebase.

Deliverables

The bonus must produce:

1. A 5–8 page architecture document
2. One high-level architecture diagram
3. At least one detailed diagram of an important system flow
4. Five or more Architectural Decision Records (ADRs)
5. Three or more identified feedback loops
6. Analysis of at least three potential bottlenecks
7. An architecture evolution analysis for 10,000 users
8. Five or more Architecture Fitness Functions
9. A critical review of the current architecture

The diagrams may be created using draw.io.

Architectural Decision Record

Each architectural decision should follow a structure similar to:

```
id="p8m2qx"
# ADR-001: Use SQLite as the persistence layer

## Context

Tache is designed as a local-first CLI application.

## Problem

The application requires persistent relational storage without
requiring the user to configure an external database server.

## Options

1. SQLite
2. PostgreSQL
3. JSON files

## Decision

Use SQLite through Drizzle ORM.

## Consequences

Positive:
- Zero database server configuration
- Relational queries
- FTS5 support
- Easy local deployment

Negative:
- Limited concurrency
- More difficult future multi-user deployment

## Future Consideration

If Tache becomes a multi-user distributed application,
PostgreSQL should be evaluated.
```

Expected Outcome

The expected result is not simply a description of the existing architecture.

The student must demonstrate the ability to reason about Tache as a collection of interdependent components.

The bonus should demonstrate that an apparently local architectural decision can produce consequences in other parts of the application.

For example:

```
id="5t7q2k"
Choosing local Ollama
|
+--> Privacy
|
+--> No API costs
|
+--> Hardware requirements
|
+--> Latency
|
+--> Distribution complexity
|
+--> Model management
```

Similarly:

```
id="4x8m1r"
Choosing Clean Architecture
|
+--> Better testability
|
+--> Replaceable infrastructure
|
+--> More abstractions
|
+--> More initial complexity
|
+--> Easier long-term evolution
```

The evaluation should prioritize the quality of architectural reasoning and the identification of trade-offs rather than simply the number of diagrams or pages produced.

Conclusion of Bonus

This bonus turns Tache itself into the subject of **Systems Thinking**.

While the main project evaluates the ability to build, test, and distribute the system, the bonus evaluates the ability to understand, question, critique, and evolve that system.

The two components therefore become complementary:

1. **Required** — build, test, document, and distribute Tache.
2. **Bonus** — analyze, critique, and architecturally evolve Tache.

The final objective is to develop not only the ability to implement software, but also the ability to make conscious architectural decisions about real-world systems.

Conclusion

This lab combines advanced TypeScript programming with practical software architecture and systems thinking.

The Tache project demonstrates how a production-oriented CLI application can combine strong typing, design patterns, SOLID principles, Clean Architecture, automated testing, persistent storage, local AI, and professional software delivery practices.

The systems-thinking component extends the exercise beyond implementation by requiring architectural decisions to be evaluated in terms of business problems, constraints, dependencies, scalability, and trade-offs.

The expected outcome is therefore not simply a working application, but a demonstration of the ability to design, implement, test, document, and reason about a complete software system.

Future work could include:

1. Multi-user synchronization
2. Remote database support
3. Distributed task processing
4. Additional AI models and providers
5. Calendar integrations
6. Plugin architecture
7. Advanced observability
8. Distributed event processing
9. Cloud deployment